

Documentation :

Log File Analyzer

1 - What is the purpose of this project?

Purpose = teaching your computer to be a detective for boring computer diaries.

Every program on a computer keeps a diary. That diary is called a log file. It writes down everything that happens:

```
"I started at 9am"  
"Oops, I couldn't find the database"  
"Warning, I'm getting too hot"
```

The problem: These diaries are HUGE. Thousands of lines. No human wants to read all that.

Your project builds a robot helper that reads the diary for you and gives you a summary.

Instead of reading 10,000 lines, you get:

Errors: 2

Warnings: 1

Info: 2

Biggest problem: Database connection failed

That's it. You turn a messy diary into a 3-line report card.

What a log file really is

A log file is a text file where an app writes down what it's doing, while it's doing it.

It includes:

1. Normal stuff: INFO User logged in, INFO Backup completed
2. Warnings: WARNING Disk space 90% full
3. Errors: ERROR Database connection failed

4. Timestamps: `2026-06-28 09:32:14` so you know *when* it happened

1.A Analyzing what kind of log?

Any text file where a program writes down what it did.

In cloud engineering, you deal with tons of these diaries. Examples:

What program writes it	What the log looks like	Why cloud engineers care
Web server like Nginx	<code>ERROR 404 /page-not-found</code>	Tells you your website is broken
Database like MySQL	<code>ERROR connection failed</code>	Tells you your app can't save data
AWS service like S3	<code>ERROR S3 bucket inaccessible</code>	Tells you your files are locked out
Your own Python app	<code>INFO Backup completed</code>	Tells you the backup actually worked
Linux system	<code>WARNING CPU usage high</code>	Tells you the server is about to crash

Your job: Count how many times `ERROR`, `WARNING`, `INFO` show up. Then find which error happened the most.

1.B Why this matters for cloud engineering

Cloud = thousands of computers running your apps. They all write diaries 24/7.

When something breaks at 3am, your boss doesn't want you reading 50,000 lines. They want: "We had 2 errors. Both were S3 bucket problems."

This project proves 4 things for your portfolio:

1. File handling: You can open and read files → basic skill for any cloud job

2. Regular expressions: You can find patterns like `ERROR` in text → used to parse real AWS logs
3. Dictionaries: You can count stuff → `{"ERROR": 2, "WARNING": 1}`
4. Error handling: Your script won't crash if the log file is missing → real-world code skill

Kid analogy:

Imagine you have a jar of mixed M&Ms. The log file is the jar. Your script is you dumping them out and saying "Ok, we have 5 red, 3 blue, 2 broken ones. Most are red."

Except instead of M&Ms, it's computer problems. And instead of eating them, you fix the broken ones.

Scripting

- **Import re**

`re` is Python's built-in regular expression library. It allows you to search for, match, and extract specific text patterns from string data using a specialized syntax. We use it here to slice up a single, chaotic string of log data into clean fields like IP, status code, and bytes.

- **From pathlib import Path**

`Path` is an object-oriented way to interact with file systems, replacing the older `os.path` string-based methods.

We use it because it handles cross-platform path differences automatically (e.g., forward slashes on Linux vs. backslashes on Windows). It also makes operations like checking if a log file actually exists (`path.exists()`) much cleaner and more readable.

- **from typing import Dict, List, Any**

These are type hints used to annotate our code. They tell both developers and automated tooling (like IDEs or linters) exactly what kind of data containers a function expects or returns.

Architecturally, this serves as explicit, living documentation. Instead of guessing what a variable holds, a type hint like `List[Dict[str, Any]]` tells us

instantly that we are dealing with a list of dictionaries where the keys are strings and the values could be any data type.

- **Parse_log_file function**

What it does: This function verifies if a file exists, opens it safely, and loops through it line-by-line to extract matches using our pre-defined regular expression. It converts numeric strings like status codes and byte sizes into proper integers, skipping lines that do not fit the expected format.

Why it exists architecturally: It acts as the serialization layer of our application pipeline. By isolating file I/O and data normalization inside this single function, the rest of our application (the analytical tools and reporting engines) remains pure and completely detached from how the raw data was stored or formatted.

Proactive Error Spotting & Review

Memory Efficiency: Notice we use `for line in file:` instead of `file.readlines()`. The `readlines()` method loads the entire file into memory at once. If your log file is 10 gigabytes, your script will crash the system. Iterating over the file pointer processes one line at a time, keeping memory usage down to virtually zero.

Data Cleansing: Standard web servers use a dash (-) when zero bytes are sent. The line `0 if entry["size"] == "-" else int(entry["size"])` handles this exception immediately so our analysis block won't crash when calculating total bandwidth.

`parse_log_file` function - Practicality

Think of this function like a mail sorter for log files.

You give it a big messy pile of log lines like `127.0.0.1 - - [27/Jun/2026] "GET /index.html" 200 1043.`

The function reads each line, picks out the important pieces like IP, time, status code, and size, and turns each line into a neat little labeled box.

At the end, it gives you back a list of all those boxes so you can count errors, find warnings, etc. later.

What is `(file_path: Path)`?

`file_path` is the address of the log file you want to read. Like telling the function "go to this folder and open this file".

`Path` means it's a special `Path` object from Python's `pathlib`. That's better than a plain string because it handles slashes, checks if files exist, and works on Windows/Mac/Linux.

So `file_path: Path` just means: "I promise to give you a `Path`, not just random text".

What does `-> List[Dict[str, Any]]` do?

This is a hint for humans and tools. It's saying "this function will return something".

`List` means a list, like a shopping list.

`Dict[str, Any]` means each item in that list is a dictionary with text keys and any type of values.

So `-> List[Dict[str, Any]]` = "You'll get back a list of dictionaries".

Example: `[{"ip": "127.0.0.1", "status": 200}, {"ip": "192.168.1.1", "status": 404}]`

It doesn't change how the code runs. It just helps you understand what comes out.

What does this block do:

Code

```
with file_path.open("r", encoding="utf-8") as file:
    for line_num, line in enumerate(file, 1):
        match = LOG_PATTERN.match(line.strip())
        if not match:
            Continue
```

```
with file_path.open("r", encoding="utf-8") as
file:
```

"Open the file to read it. Use UTF-8 so weird characters don't break it. And automatically close it when I'm done, even if there's an error." The `with` is your safety net.

```
for line_num, line in enumerate(file, 1):
```

"Go through the file one line at a time. Also count which line I'm on,

starting at 1 instead of 0."

So first loop: `line_num=1, line="first log line"`

1. `match = LOG_PATTERN.match(line.strip())`
"Take the line, remove spaces at the ends, then check if it looks like a proper Apache log line using our regex blueprint."
2. `if not match: continue`
"If this line doesn't match the pattern, it's probably broken or different. Don't crash. Just skip it and try the next line."

What does `entry = match.groupdict()` do?

If the line DID match, `match.groupdict()` grabs all the named pieces from the regex and puts them in a dictionary.

Remember your regex had `(?P<ip>\S+)` and `(?P<status>\d{3})`? Those names become keys.

So `entry` becomes something like:

```
{"ip": "127.0.0.1", "timestamp": "27/Jun/2026:12:01:02 +0000",  
"method": "GET", "path": "/index.html", "status": "200",  
"size": "1043"}
```

But notice: `status` and `size` are still text strings right now.

1.E - What does this do:

Code

```
entry["status"] = int(entry["status"])  
entry["size"] = 0 if entry["size"] == "-" else int(entry["size"])
```

This is fixing the types so we can do math later.

1. `entry["status"] = int(entry["status"])`
"The status was '200' as text. Turn it into the number 200." Now you can do `if status > 400` later.

```
2. entry["size"] = 0 if entry["size"] == "-" else  
   int(entry["size"])
```

"In logs, a dash - means 'no size' or 0 bytes. If size is '-', make it 0. Otherwise turn the text into a number."

So "1043" becomes 1043, and "-" becomes 0.

1.F - What does `parsed_logs.append(entry)` do?

`parsed_logs` is your empty shopping list from the start.

`append(entry)` means "take this neat dictionary we just made and stick it at the end of the list".

Do this for every good log line. At the end, `parsed_logs` has all the entries.

1.G - What does `return parsed_logs` do?

"I'm done sorting. Here's the full list of all the dictionary boxes I made. Do whatever you want with them."

This sends the list back to whoever called `parse_log_file`. Next, your `analyze` function can count errors from it.

Quick note on your project design:

You're on the right track with "parsing, analyzing, and reporting are decoupled". This `parse_log_file` is just step 1: the parser. Next you'd build `count_errors(parsed_logs)` and `report_results(counts)`. That modular setup is exactly what production code looks like.

Next function we introduced was the `analyze_log_data`

Python

```
from collections import Counter

def analyze_log_data(parsed_logs: List[Dict[str, Any]]) -> Dict[str, Any]:
    if not parsed_logs:
        return {"total_requests": 0, "top_ips": [], "top_paths": [], "error_rate": 0}

    total_requests = len(parsed_logs)
    total_bytes = sum(entry["size"] for entry in parsed_logs)

    ip_counter = Counter(entry["ip"] for entry in parsed_logs)
    path_counter = Counter(entry["path"] for entry in parsed_logs)

    # Calculate error rate (status codes >= 400 indicate client/server errors)
    error_count = sum(1 for entry in parsed_logs if entry["status"] >= 400)
    error_rate = (error_count / total_requests) * 100

    return {
        "total_requests": total_requests,
        "total_bytes": total_bytes,
        "top_ips": ip_counter.most_common(5),
        "top_paths": path_counter.most_common(5),
        "error_rate": round(error_rate, 2)
    }
```

What it does: This function accepts the list of parsed log dictionaries and aggregates them. It uses `Counter` to track the frequency of IP addresses and paths, sums up the data transfer payload sizes, and calculates what percentage of requests resulted in an HTTP error status code.

Why it exists architecturally: This is the pure business logic layer. It has zero knowledge of files, regex, or terminal output format. It only cares about processing structured data in memory, making it highly reusable and trivially easy to unit test.

Now is the implementation of the second function :

Yep, let's switch from "age 5" to "junior engineer" mode. Same code, less toys, more why.

`analyse_log_data` - Beginner Engineer Version

```
def analyse_log_data(parsed_logs: List[Dict[str, Any]]) -> Dict[str, Any]:
```

What: Function takes a list of parsed log entries and returns a dict of stats.

Why: Type hints `List[Dict[str, Any]]` make the contract clear. Your IDE and mypy can catch you passing the wrong type.

```
if not parsed_logs:
```

```
    return {"total_requests": 0, "top_ips": [], "top_paths": [], "error_rate": 0.0, "total_bytes": 0}
```

What: Early return if input is empty.

Why: Prevents `ZeroDivisionError` later and avoids crashes. This is called a "guard clause". Always handle the empty case first.

```
total_requests = len(parsed_logs)
```

What: Count total log entries.

Why: We need this for percentage calculations. `len()` is $O(1)$ on lists, so it's free.

```
total_bytes = sum(entry["size"] for entry in parsed_logs)
```

What: Add up the `size` field from every entry.

Why: Gives total bandwidth served. Generator expression `(...)` is memory efficient vs building a list first.

Gotcha: Relies on `parse_log_file` already converting `size` to `int`. If it were still a string, this throws `TypeError`.

```
ip_counter = Counter(entry["ip"] for entry in parsed_logs)
```

```
path_counter = Counter(entry["path"] for entry in parsed_logs)
```

What: Count frequency of each IP and each URL path.

Why: `collections.Counter` is the standard way to do frequency counts. Cleaner and faster than manual `dict` loops.

```
error_count = sum(1 for entry in parsed_logs if entry["status"] >= 400)
```

```
error_rate = (error_count / total_requests) * 100
```

What: Count HTTP errors and convert to %.

Why: HTTP spec: 4xx = client errors, 5xx = server errors. `total_requests` is safe here because we already returned if the list was empty.

Note: This counts 3xx redirects as "not errors". That's correct for most dashboards.

```
return {  
  
    "total_requests": total_requests,  
  
    "total_bytes": total_bytes,  
  
    "top_ips": ip_counter.most_common(5),  
  
    "top_paths": path_counter.most_common(5),  
  
    "error_rate": round(error_rate, 2)  
  
}
```

What: Package all metrics into one dict.

Why: `most_common(5)` gives top 5 without you sorting manually. `round(..., 2)` keeps output readable. Returning a dict makes it easy to convert to JSON/CSV later.

Biggest issue in your script right now: This function expects Apache-style logs with `ip, path, status`. Your example `INFO User logged in` has none of those fields, so `entry["ip"]` would `KeyError`.

The Report Generator function

Python



```
def display_report(metrics: Dict[str, Any]) -> None:
    if metrics["total_requests"] == 0:
        print("=== Log Analysis Report ===")
        print("No valid log entries found to analyze.")
        return

    print("=== Log Analysis Report ===")
    print(f"Total Requests Processed : {metrics['total_requests']}\n")

    # Format bytes to a human-readable Megabyte format
    mb_transferred = metrics["total_bytes"] / (1024 * 1024)
    print(f"Total Data Transferred   : {mb_transferred:.2f} MB")
    print(f"HTTP Error Rate              : {metrics['error_rate']}%\n")

    print("Top 5 Most Active IP Addresses:")
    for ip, count in metrics["top_ips"]:
        print(f"  - {ip}: {count} requests")

    print("\nTop 5 Most Requested Paths:")
    for path, count in metrics["top_paths"]:
        print(f"  - {path}: {count} requests")
    print("=" * 27)
```

What it does: This function takes the final dictionary of aggregated metrics and prints a structured, human-readable summary report to the terminal, including data volume conversions and top-five distributions.

Why it exists architecturally: This represents the presentation layer of the application. By separating output formatting into its own module, you ensure that transitioning the tool in the future—such as generating an HTML report, sending a JSON payload to an API, or writing to a CSV file—requires changing only this presentation function, leaving your core analytical and parsing engines completely untouched.

display_report - Beginner Engineer Version

```
def display_report(metrics: Dict[str, Any]) -> None:
```

What: Function takes a metrics dict and returns nothing None. It just prints.

Why: Type hint `Dict[str, Any]` matches what `analyse_log_data` returns. -> None tells you + IDE this function is for side effects only, not for computing values.

Pattern: Separating analysis from display = good architecture. You can test analysis without printing, and change printing without touching math.

```
if metrics["total_requests"] == 0:
    print("=== Log Analysis Report ===")
    print("No valid log entries found to analyze.")
    return
```

What: Guard clause. If there were zero requests, print a short message and exit early.

Why: Avoids printing empty tables like "Top 5 IPs:" with nothing under it. `return` stops the function so the rest doesn't run. User experience win.

Gotcha: Uses `metrics["total_requests"]` directly. If someone passes a dict without that key, you get `KeyError`. Defensive coding would use `metrics.get("total_requests", 0)`.

```
print("=== Log Analysis Report ===")
print(f"Total Requests Processed : {metrics['total_requests']}\n")
```

What: Print header + total count. `\n` adds a blank line after for spacing.

Why: `f"..."` is an f-string. It drops variables directly into text. Cleaner than `"Total: " + str(metrics['total_requests'])`.

```
mb_transferred = metrics["total_bytes"] / (1024 * 1024)
print(f"Total Data Transferred : {mb_transferred:.2f} MB")
```

What: Convert bytes to megabytes. $1024 * 1024$ = bytes in 1 MB.

Why: Raw bytes like 10485760 mean nothing to humans. 10.00 MB does.

`:.2f` = format specifier: round to 2 decimal places, force float display. 10 becomes 10.00.

Anti-pattern avoided: Not doing `round(mb_transferred, 2)` because f-string formatting is cleaner for display.

```
print(f"HTTP Error Rate : {metrics['error_rate']}%\n")
```

What: Print error percentage from your analysis.

Why: \n again for visual separation. You already rounded in analyse_log_data, so no need to round here.

```
print("Top 5 Most Active IP Addresses:")
for ip, count in metrics["top_ips"]:
    print(f" - {ip}: {count} requests")
```

What: metrics["top_ips"] is a list like [('192.168.1.1', 42), ('10.0.0.1', 30), ...] from .most_common(5).

Why: for ip, count in ... unpacks each tuple directly. No need for item[0] and item[1]. Pythonic and readable.

Structure note: The function assumes top_ips exists and is iterable. If analyse_log_data ever changed its output, this breaks. That's why type hints + tests matter.

```
print("\nTop 5 Most Requested Paths:")
for path, count in metrics["top_paths"]:
    print(f" - {path}: {count} requests")
```

What: Same pattern, but for URLs instead of IPs.

Why: DRY principle - you could refactor both loops into a helper print_top_5(title, data) later if you wanted.

```
print("=" * 27)
```

What: Prints ===== as a footer.

Why: "=" * 27 repeats the string 27 times. Visual separator so the report doesn't run into your next terminal command. 27 is arbitrary - matches the width of "=== Log Analysis Report ===".

Key takeaways for reading real code

Pattern	Why it matters
Guard clause at top	Fails fast, avoids cluttered output on empty data
f-strings with <code>:.2f</code>	Standard way to format numbers for humans
Tuple unpacking in <code>for</code> loop	Cleaner than indexing: <code>ip, count</code> vs <code>item[0], item[1]</code>
Function does one thing	Only prints. No math, no file I/O. Easy to test/replace

Subtle bug risk

If `analyse_log_data` ever returns `top_ips: []`, the loop just prints nothing. That's fine. But if it returns `top_ips: None`, this crashes with `TypeError: 'NoneType' object is not iterable`.

How you'd spot it: Read the contract. `analyse_log_data` promises `List`, not `None`. `mypy` would catch it if you broke that contract.

Next step in a real repo: you'd add a `main()` function that does `logs = parse_log_file(...)` → `metrics = analyse_log_data(logs)` → `display_report(metrics)`. That's the full pipeline.

The `main()`function

```
def main() -> None:
    # Example local path setup: points to a sample file named 'access.log'
    log_file = Path("access.log")

    try:
        print(f"Initializing scan on: {log_file}...")
        raw_data = parse_log_file(log_file)
        analysis_results = analyse_log_data(raw_data)
        display_report(analysis_results)

    except FileNotFoundError as e:
        print(f"Configuration Error: {e}")
    except Exception as e:
        print(f"An unexpected system error occurred during execution: {e}")

if __name__ == "__main__":
    main()
```

What it does: This is the controller function (main) that manages the operational execution lifecycle. It defines the target path, triggers the sequential data pipeline (Read $\rightarrow$ Analyze $\rightarrow$ Display), and acts as a safety net catching runtime errors.

Why it exists architecturally: The `if __name__ == "__main__":` guard block prevents execution when this file is imported as a module by another script. The centralized try/except block wraps the entire pipeline here, ensuring that errors are intercepted gracefully at the highest level instead of letting individual worker functions crash uninformatively.

```
def main() -> None:
```

What: Defines the main function. Returns nothing.

Why: Convention. `main()` is where your program starts its real work. Keeping logic in a function vs. global scope makes it testable and import-safe. `-> None` = no return value, just runs stuff.

```
log_file = Path("access.log")
```


What: Creates a `Path` object pointing to a file called `access.log` in the current folder.

Why: `pathlib.Path` is the modern way to handle file paths. Safer than raw strings `"access.log"` because it handles `/` vs `\` on Windows/Mac/Linux.

Hard-coded path: In real code you'd get this from command line args. This is fine for learning.

```
try:
```

What: Start a "try block". Python will attempt everything indented under here.

Why: File I/O + parsing can crash. `try/except` lets you catch errors gracefully instead of dumping a scary traceback to the user.

```
print(f"Initializing scan on: {log_file}...")
```

What: Print status message before doing work.

Why: User feedback. If parsing 2GB of logs takes 30s, the user knows the program didn't freeze. UX matters in CLI tools.

```
raw_data = parse_log_file(log_file)
```

What: Call your parser. Turns the text file into `List[Dict]`.

Why: This is step 1 of your pipeline: Parse. If the file doesn't exist, this line raises `FileNotFoundError` and jumps to `except`.

```
analysis_results = analyse_log_data(raw_data)
```

What: Call your analysis function. Turns `List[Dict]` into summary `Dict`.

Why: Step 2: Analyze. Notice the separation: `parse_log_file` doesn't know about stats, `analyse_log_data` doesn't know about files.

Typo alert: Your earlier function was `analyse_log_data` with `s`. Here you wrote `analyze_log_data` with `z`. Python will `NameError` if they don't match. Pick one spelling.

```
display_report(analysis_results)
```

What: Call your print function. Takes the summary dict and prints it.

Why: Step 3: Present. Your full pipeline is `Parse -> Analyze -> Present`. Clean separation.

```
except FileNotFoundError as e:  
    print(f"Configuration Error: {e}")
```

What: If `parse_log_file` raised `FileNotFoundError`, catch it here and print a friendly message.

Why: Specific exception first. Better than "Something broke". `as e` gives you the actual error message: `Target log file not found at: access.log`.

Pattern: Catch expected errors specifically.

```
except Exception as e:  
    print(f"An unexpected system error occurred during execution: {e}")
```

What: Catch *any other* error that wasn't `FileNotFoundError`.

Why: Safety net. If `analyse_log_data` gets a `KeyError` because a dict is missing status, this catches it so the program doesn't crash ugly.

Anti-pattern risk: `except Exception` is broad. In production you'd log the full traceback. For learning, this is fine.

`if __name__ == "__main__":` - The magic line

```
if __name__ == "__main__":  
    main()
```

What: "Only run `main()` if this file is executed directly, not if it's imported."

Why: Two scenarios:

How you run it	What <code>__name__</code> equals	Does <code>main()</code> run?
<code>python your_script.py</code>	<code>"__main__"</code>	Yes
<code>from your_script import analyse_log_data</code>	<code>"your_script"</code>	No

Why it matters: Without this guard, importing your file to use `analyse_log_data` in another script would accidentally run the whole log scan. That breaks reusability.

Rule: Every script you intend to run should have this block. Every library/module should NOT.

Big picture: What you just built

```
main()  
    └─> parse_log_file()      # Input layer  
    └─> analyse_log_data()    # Logic layer  
    └─> display_report()      # Output layer
```

This is called a "3-tier architecture" in tiny form. Real-world tools like `pytest`, `black`, `flake8` all use this pattern.

One bug to fix: You wrote `analyze_log_data` with `z` but defined `analyse_log_data` with `s`. Pick British or American spelling and be consistent, or Python throws `NameError`.